# AERIAL v2 in plain language

The same project as the first tab, explained from first principles. Technical terms are kept, but each one is introduced before it is used.

## The story in two minutes

After an earthquake, ten drones are sent into a damaged city district to find 16 trapped people. There is no control tower: each drone decides for itself and can only talk to drones within about 35 m. Radio packets get lost 10% of the time. And one of the ten drones has been compromised: it pretends to search an area, reports "nobody here", and broadcasts a fake position so nobody notices it is actually hovering somewhere else.

The standard textbook defence against a bad teammate is **filtering**: when you combine numbers from neighbours, throw away the most extreme ones before averaging (this is W-MSR, explained below). Filtering protects the *average*, but it has two blind spots:

- A careful liar keeps its numbers inside the normal range, so it is never filtered out.
- Some of the most damaging lies are not numbers at all. "I searched this block and it's empty" cannot be averaged with anything.

So the project adds a second layer on top of filtering: each drone tracks what it actually **knows** (things that must be true no matter who the liar turns out to be) separately from what it merely **believes** (things someone told it). It only takes risky actions, like flying into a gap or declaring an area searched, on knowledge.

The one-line thesis: *the filtering that makes a fleet resilient also makes it ignorant; we measure that trade-off.*

The experiment compares fleets that are identical except for how much they trust what they hear:

- **naive**: filters numbers, but believes every claim;
- **gated**: filters, checks claims against physics, audits teammates, and only moves when it knows the move is safe;
- **oracle**: secretly told who the liar is from the start (the best case);
- **unfiltered**: no defence at all (the worst case).

In the demo run on the slides, the naive fleet found 10 of 16 people and had 6 near-collisions; the gated fleet found all 16 with none, and every honest drone knew who the liar was within about 13 s.

# How the simulation is put together

Think of the code as two clearly separated roles:

- **The simulator** plays the role of reality. It alone knows where every drone truly is, what the terrain looks like, and where the survivors are. (`aerial/core/simulator.py`)
- **The agents** are the drones' onboard computers. Each one receives only what its own sensors and radio would give it, and returns a command. (`aerial/agents/mission.py`)

This separation is enforced by a test: if any drone code ever imports the ground-truth world module, the test suite fails. That matters because a simulation where drones can secretly peek at the truth would make every result meaningless.

## One tick = 0.1 s

The simulation advances in fixed steps of 0.1 s, for up to 240 s (2,400 ticks). In every tick, in this order:

1. **Every drone announces itself.** It produces a *beacon*: "I am at (x, y, z), moving with velocity v." Honest drones tell the truth; the liar may not.
2. **Sensors fire.** Five times a second, each drone gets a camera frame. Every tick, each pair of drones in contact also gets a *UWB range*: a radio-based distance measurement (ultra-wideband, the same technology used in item trackers). The key property: the range is measured physically, so the liar cannot fake it.
3. **Messages are delivered.** The simulator decides who can hear whom (within 35 m, minus random 10% losses) and delivers beacons and relayed messages.
4. **Every drone decides.** Each one looks at its own inbox and sensors and returns an acceleration command.
5. **Physics moves everyone.**
6. **Everything is recorded** for metrics and plots.

Why measure everything at the same instant: if the beacon were from one moment and the range from another, an honest drone that was simply moving could look like it lied. Fixing the order removes that excuse and that risk.

## The drone's physics

Each drone is modelled as a **double integrator**: the command is an acceleration, which changes velocity, which changes position. Horizontal speed is capped at 5 m/s, vertical at 2 m/s (multirotors climb slower than they cruise), and acceleration at 4 m/s². There are no aerodynamics; the course is about coordination and reasoning, not flight dynamics, and the proposal said so.

### Fair comparisons: paired randomness

Randomness (packet losses, sensor noise) comes from separate, named random streams created from one seed. Each stream draws the same amount every tick no matter what the drones do. So when the naive and gated fleets fly the same seed, they experience *exactly* the same lost packets and the same noise. The runner checks this and refuses to report a comparison if the two runs drew different random numbers. Without this, "gated did better" might just mean "gated was luckier".

### Configuration

Every parameter lives in one typed config (`aerial/core/config.py`) that can be overridden from scripts, e.g. `"comms.R=30"`. Before a run, the config is validated, including some geometry checks: for example, the minimum spacing drones keep from each other must be larger than the safety margin the gate uses, or two honest drones would block each other forever.

## The world and what a drone can sense

### The city

The terrain is a **height map**: the arena (120 × 120 m) is cut into 1 m squares, and each square has one height. City blocks about 20 m wide are separated by 8–12 m streets. Buildings are 12–35 m tall, so many are taller than the drones' 15 m ceiling and must be flown *around*. 40% of buildings have collapsed into rubble piles 1–5 m high (some piles reach 13–18 m), with debris spilling into the street.

A height map is called "2.5D" because it can't represent overhangs, like a bridge. It was chosen because questions such as "can I see that point?" or "is it safe to fly at 9 m here?" become simple lookups.

The search zone is the central 100 × 100 m, divided into 2,500 cells of 2 × 2 m. A cell is *covered* once some drone has looked at it properly. The 16 survivors are only placed where a drone could actually see them, so no fleet is penalised for missing someone hidden under a roof.

One drone is chosen as the liar, and a 50 × 50 m corner of the zone is the area it will pretend to have searched. In the demo map, three survivors are deliberately placed deep inside that corner, so believing the lie costs lives.

### The camera

Each drone has one forward-facing camera, tilted 30° down, with a 90° × 60° field of view. It's simulated by casting 384 rays (32 across × 12 down) up to 25 m. From each frame a drone learns:

- **terrain facts**: "the ground here is lower than this point" (a ray passed over it) and "the ground here is at least this high" (a ray hit it);
- **which cells it saw**: a cell counts as seen if it's within 15 m, at least 20° below the camera, inside the view, and not blocked;
- **survivors**: anyone in a seen cell is detected, with a noisy position, a body-heat reading and a yes/no "saw movement";
- **other drones**: any drone inside the volume the camera can see.

The most important design choice in the whole sensing layer: **one function decides "seen" for both coverage and survivor detection.** If a cell was seen, any survivor in it *was* detected. That means "I looked at this cell and nobody was there" is a hard fact, not a guess. Later, this is what lets an audit *prove* that a teammate lied about searching a cell.

Similarly, the camera keeps a record of which space it has confirmed empty. If a drone claims to be at a spot that my camera can see is empty, that claim is provably false.

Only one thing stays genuinely uncertain: whether a found survivor is alive or dead. Heat and movement are noisy, so that's handled as a belief (see the epistemic section).

## How drones talk to each other

### Who can hear whom

Drone i hears drone j in a given tick if they are within 35 m **and** the packet isn't lost (90% chance it gets through). Losses are independent in each direction, so i might hear j while j misses i, which is why the code treats the network as a directed graph.

If a drone is waiting for information it needs, it can **re-query**: in the next tick it gets a second, independent chance to hear each nearby drone.

### Two kinds of traffic

**Beacons** are short status messages ("I'm here, moving like this") sent every tick and heard only by direct neighbours. Because every drone beacons every tick, *silence is informative*: a drone that hasn't been heard for 9 ticks in a row is almost certainly out of range (the chance of losing 9 packets in a row at 10% loss is one in a billion). This "silence rule" is the only probabilistic assumption in the safety reasoning.

**Events** are news that should reach the whole fleet: "I cleared these cells", "survivor here", "I have proof drone 4 lied", terrain updates, range measurements. They spread by **gossip**: each drone forwards new events to its neighbours once, one hop per tick, and ignores duplicates. News crosses the fleet in a few ticks even when most drones can't hear each other directly.

Bandwidth is limited to 64 events per drone per tick, so events are prioritised: accusations first (knowing who the liar is changes how you treat everything else), then survivors, then coverage, and bulk terrain and range data last.

### What the liar can and can't do with messages

Each event is stamped with its true sender by the simulator, not by the drone. This models digitally signed messages without implementing cryptography. So the liar can lie about *itself* and can refuse to forward others' messages, but it can't impersonate another drone. Messages are broadcast, so the liar also can't tell different stories to different neighbours.

Why these restrictions: if the liar could forge messages from honest drones, it could frame them, and no system could guarantee never accusing an innocent drone.

## How a drone remembers and plans

### Every claim remembers who made it

This is the single most useful idea in the code. A drone never stores just "cell 812 is searched". It stores "cell 812 is searched **according to drones 2 and 5**", plus a separate flag for "I saw it myself".

The "according to" part is stored compactly as a **bitmask**: a 64-bit number where bit j is 1 if drone j vouched for the claim. Checking "did anyone other than drone 4 vouch for this?" becomes one fast bitwise operation over all 2,500 cells at once.

Why it pays off:

- When the liar is caught, every claim that rested *only* on its word automatically goes back to "not searched". Nothing has to be undone by hand.
- The naive and gated fleets share exactly the same maps; they just read them with different rules ("believe anyone" vs "believe only what's proven"). That keeps the comparison fair.

### The terrain map: ranges, not guesses

Drones start with **no map** at all. For each 1 m square they keep a range: "the ground here is somewhere between *lo* and *hi* metres", starting at 0 to infinity.

- A camera ray passing over the square lowers *hi*.
- A ray hitting something there raises *lo*.

A square is safe to fly over at a given altitude only if *hi* plus a safety margin is below that altitude. Why ranges instead of a single best estimate: if a ray hits a wall at 8 m, all you know is that the wall is *at least* 8 m tall. Recording "height = 8" and flying over at 9 m could fly you into a 20 m building. Unknown squares are never treated as safe.

Terrain reported by other drones goes through the same trust rules as everything else, so the liar can't lure drones into a building by claiming it's open air.

### Planning a route

- The planner works on a grid of 2 m cells, each either known-free at the drone's altitude or not.
- It finds shortest routes with **Dijkstra's algorithm** (via SciPy, which runs it in fast compiled code). Diagonal moves are only allowed if both neighbouring cells are free, so routes never clip a building corner.
- The step-by-step grid route is then simplified into straight segments wherever the straight line stays on free cells.

### Dividing the work: a Voronoi partition

To avoid all ten drones searching the same street, each drone claims the cells that are closer to it than to any other drone. That's a **Voronoi partition**. Here, "closer" is measured along actual flyable paths rather than in a straight line (a *geodesic* Voronoi): a cell 10 m away but behind a building block might really be 60 m away.

Each drone recomputes this once per second from everyone's last known positions. When it runs out of work in its own region, it takes uncovered cells from anywhere (work stealing). When the liar is caught, it's simply dropped from the calculation, and its region is shared out among the others automatically.

## The four fleets and the mission flow

### One drone program, four trust settings

All four fleets run the **same** drone code. A fleet is just a different setting of four switches:

| Fleet | Whose claims it acts on | How it combines numbers | Checks for lies | Audits teammates | Waits until a move is known safe |
|---|---|---|---|---|---|
| unfiltered | everyone, even proven liars | plain average | no | no | no |
| naive | anyone not proven a liar | W-MSR filter | no | no | no |
| gated | anyone, to share out work; only proven facts to judge what's done | W-MSR filter | yes | yes | yes |
| oracle | knows the liar from the start | W-MSR filter | not needed | no | yes |

This is a controlled experiment: since everything else is identical, any difference in outcome is caused by trust alone.

## What a drone does over a mission

Each drone moves through a small set of modes:

1. **Transit**: fly from the launch strip to the edge of the search zone in a loose line. Drones adjust their speed so nobody runs far ahead, using consensus on how far forward each one is (W-MSR or plain average, depending on the fleet).
2. **Explore**: pick the best unsearched cell in my region, plan a route, fly there, look. Repeat.
3. **Confirm**: when a survivor is spotted, hover and keep looking until there's enough evidence to call them alive or dead (at most 8 s).
4. **Audit** (gated only): when nothing is left to search, re-check a random sample of each teammate's "searched" claims.
5. **Return** to the launch strip, then **Landed**.

If new work appears at any point, for instance because the liar was just caught and its "searched" area reopened, the drone drops back into Explore. That's how the re-sweep happens with no special-case code.

## Small behaviours that matter

- **Three altitude layers.** Drones are assigned 6, 9 or 12 m by their index, which keeps many crossing paths vertically apart.
- **Turn before you move.** If a drone needs to turn more than 45°, it rotates in place first. The camera faces forward, so this guarantees the drone has looked at where it's about to go, which both the terrain map and the safety gate depend on.

- **Safety reflexes are never blocked.** Pushing away from a drone closer than 3 m, from known walls, and from the arena edge happens unconditionally. Braking can't make things worse, so it never waits for permission.
- **Stable target choice.** A drone keeps its current target unless another is at least 6 m better. Without this it would flip between two targets forever.
- **Staggered replanning.** Each drone replans every 4 s plus a small offset based on its id, so ten drones don't all compute routes in the same tick.
- **Giving up gracefully.** A cell that still can't be seen after 3 attempts is marked abandoned, so the mission can actually end.

### Audits: spot-checking teammates

When the gated fleet runs out of work, each drone revisits a random 10% (the audit fraction q) of each teammate's "searched" cells that nobody else has looked at. Picking cells uniformly at random is what makes this meaningful: if a teammate falsely claimed a cell with a survivor in it, a random 10% sample has at least about a 10% chance of landing on that cell, whatever the liar chose to lie about. Raise q and you catch more lies but finish later. That's why q is called the caution knob.

### Pinning down survivor positions

Each drone's position estimate for a survivor is noisy. Drones share estimates and combine them using the fleet's method. The project's localisation study shows the trade-off clearly. Plain averaging over repeated rounds gets dragged almost the full distance of a lie (2, 8 or 20 m), because a stubborn liar keeps pulling the average towards its value. W-MSR stays within about 0.3–0.7 m of the truth. But when nobody lies, W-MSR is slightly *worse* (0.2–0.45 m error against roughly 0 for the average), because it throws away some honest data.

## W-MSR filtering and how the liar lies

### W-MSR, intuitively

Suppose you and your neighbours each hold a number (say, how far forward you are) and you want to move towards the group's value. With a plain average, one neighbour reporting a huge number drags everyone along. W-MSR (Weighted Mean-Subsequence-Reduced, LeBlanc et al. 2013) fixes this by assuming at most F neighbours can lie:

1. Look at the values **above** yours. Throw away the F highest.
2. Look at the values **below** yours. Throw away the F lowest.
3. Average what's left, together with your own value.

Why it's safe: if a lying value survives the trimming, there must be a trimmed value even further out, and at least one of those trimmed values came from an honest drone. So whatever survives lies within the range of honest values. A liar can nudge the result, but can't drag it outside what honest drones are reporting.

Two limits, which are the whole reason for the project:

- A careful liar that reports something *within* the honest range is never trimmed, so it is never detected.
- W-MSR works on numbers. A statement like "I searched this block" has nothing to average.

The guarantee also depends on the network: every group of drones needs enough links to the rest of the fleet. The formal condition is called **(r, s)-robustness**. The code can check it for up to 20 drones in about a second, using a bitmask trick that avoids trying all 3^N pairs of drone groups. The project's random-graph study confirms the theory: when the condition holds, W-MSR succeeded in all 1,428 runs; when it didn't, only 32% of 1,002 runs succeeded.

## The liar

The liar runs the same code as an honest drone, with the same sensors and no secret knowledge. For the first 12 s it behaves honestly, so it isn't caught before the mission starts. Then it does two things.

**Fakes its search.** It stops searching, hovers, and broadcasts "searched, nobody here" for cells in its target corner. It never reports any survivor it sees.

**Fakes its position.** It needs everyone to believe it's flying a search pattern over that corner. But every listener measures its true distance by UWB, so a careless fake position gets caught straight away. The default strategy (`range_aware`) is cleverer: the liar knows its true position and its listeners' positions, so it computes which fake positions would produce exactly the distances each listener will measure.

- With one or two listeners, many such fake positions exist, so the lie survives the distance check.
- With three or more listeners spread out, the only position matching all the distances is the true one. This is trilateration, the same principle GPS uses. The liar has to fall back to the truth.

So a lying drone must avoid company. With `seek_quiet` turned on (the default), the liar first drifts away until at most two drones can hear it, and only then starts lying.

For the slides, the demo uses a blunter liar: it claims its whole corner at once (`claim_all`) and doesn't wait for a quiet spot. That distinction matters for how you present results; see the last section.

## Knowing vs believing: the epistemic layer

"Epistemic" means "about knowledge". This layer is what makes the project more than a W-MSR simulation.

### Possible worlds, made concrete

The formal idea from epistemic logic: an agent **knows** something if it's true in every situation the agent can't rule out. These situations are called *possible worlds*.

In this project, a possible world is a guess about who the liars are. With 10 drones and at most one liar (F = 1), drone 3 has 10 possibilities: "nobody lies", or "drone 0 lies", "drone 1 lies", and so on, skipping itself (it knows it's honest). With 20 drones and F = 2, there are 191 possibilities. Each drone keeps this list and crosses entries off as evidence arrives.

From the list it reads off:

- **I know j is the liar** if j is guilty in every possibility left;
- **I know j is honest** if j is guilty in none of them;
- **j might be the liar** if j is guilty in at least one.

This logic is called S5: the standard logic of knowledge, where what you know is exactly what holds across everything you can't tell apart.

### Worked example

I'm drone 3. Drone 5 broadcasts "I have proof drone 0 lied". I can't verify drone 5's proof, so the honest conclusion is "either 0 lied, or 5 is lying about 0". I cross off everything except {0} and {5}. I now **know** drones 1, 2, 4, 6, 7, 8 and 9 are honest.

Then my own camera catches drone 0 claiming to be somewhere I can see is empty. Only {0} survives. I know 0 is the liar, and because at most one drone lies, I know **everyone else is honest**. At that moment, every claim any other drone has made becomes knowledge for me.

### How lies get caught

Each check is chosen so that **an honest drone can never fail it**, which is why the system never accuses an innocent drone:

| Check | Plain meaning |
|---|---|
| R1 distance | Your claimed position doesn't match the distance I just measured to you |
| R2 speed | Your claimed movement is faster than any drone can fly |
| R3 radio | You claim to be out of radio range, yet I can hear you |
| R4 camera | You claim to be in a spot my camera shows is empty |
| R5 audit | I found a survivor in a cell you said you'd searched and found empty |
| R6 shared distance | Another drone's measured distance to you contradicts your claim, so one of you lied |

R1–R5 prove exactly who lied. R6 only proves that one of two drones lied, so it removes possibilities rather than convicting anyone outright.

One honest caveat: R1 relies on distance-measurement noise never exceeding 0.2 m. The simulation clips the noise so that's always true. That's a modelling assumption, and real radios occasionally produce outliers.

## Two standards of trust

A claim such as "cell 812 searched by drones 2 and 5" can be judged two ways:

- **Belief**: someone not yet proven a liar said so.
- **Knowledge**: I saw it myself, or in every remaining possibility at least one of the drones vouching for it is honest.

The gated fleet hands out work on belief, so drones don't re-search everything themselves, and reports how much is *known* searched. The gap between "believed searched" and "known searched" is, quite literally, the ignorance the thesis talks about.

## Alive or dead: a belief, deliberately

Whether a survivor is alive can never be certain: body heat and movement are noisy and overlap. So this is handled as a probability, using Wald's **sequential probability ratio test (SPRT)**. Each look adds evidence (warm and moving points to alive; cool and still points to dead), and the drone decides once it is 95% confident either way. The SPRT is used

because it needs the fewest looks on average for a given error rate, so drones hover only as long as necessary. Having both a belief and real knowledge in the same system lets the report show the difference clearly.

### The safety gate

Before each move, a gated drone asks: **do I *know* this move is safe?** It imagines the next tick of motion followed by braking, and checks every nearby drone:

- If the neighbour might be honest, it is near where it claims, give or take how far it could have accelerated since. Stay at least 1 m clear of that zone.
- If the neighbour might be the liar, its claim means nothing, but the measured distance doesn't lie. It's somewhere on a sphere around me at that distance. If that sphere is too far away to matter, fine. Otherwise my camera must have seen that part of space empty.

If the answer is no, the drone brakes and re-queries. If two drones block each other for more than 1 s, the one with the higher id backs off. After 3 s, the drone gives up that target and picks another.

The gate reasons about these zones directly instead of listing every possible position, which would be infinitely many. For the report, there's one known gap (also in the weak-spots section of the first tab): when the possible liar's latest packet was lost, the gate currently skips it rather than using its last known distance.

## The dashboard

The dashboard (`dashboard/app.py`) is a web page built with Streamlit that runs two fleets on the same seed and shows them side by side. It is meant to be the live demo during the presentation. Run it with `make dashboard` after installing the dashboard extras.

**Left panel: choose the scenario.** You can set the map, the number of drones, F, the number of actual liars, the packet loss rate, the radio range, the camera width (90° or 150°), the liar's position strategy, the audit fraction q, which pair of fleets to compare, and the seed. Pressing *run* simulates both fleets and saves the result to disk, so opening the same settings again is instant. A new setting takes a few minutes.

**Five tabs:**

- **Mission**: an animated 3D view per fleet, with buildings, drones and their trails, camera view lines, and survivors (red until found, then green or grey for alive or dead). The liar is red and turns orange once caught. Below each view: survivors found, completion time, collisions, people missed in the liar's area, and when everyone knew the liar.

- **Knowledge**: a grid showing which drone knows who the liar is at a chosen moment; curves of believed vs known coverage; the share of drones that know the liar over time, with an epidemic-model (SI) curve fitted to it; and a timeline of when each drone's gate was open.
- **Safety**: the STL safety score over time, explained in the next section. Above zero means safe so far.
- **Coordination**: how far drones drift out of line during transit.
- **Batch**: a scatter of completion time against survivors missed for every run of a trade-off sweep, coloured by fleet.

**Before presenting, fix or know these:**

- The left panel has no switch for the blunt liar the slides use. With the default careful liar, naive may well find everyone, so the live comparison could look unimpressive. Adding the two switches is quick.
- The Batch tab will be empty, because the v2 trade-off sweep hasn't been run yet.
- "Actual liars" is capped at F, so you can't demonstrate more liars than assumed.
- If you change settings without pressing *run*, the old results stay on screen.
- Run every setting you plan to show ahead of time; nobody wants to wait three minutes on stage.

# Experiments, tests, and what is still shaky

## Grading a run with STL

**Signal Temporal Logic (STL)** is a way of writing requirements over time, like "always keep 1 m from every other drone" or "eventually find survivor 7". Its useful feature is a **robustness score**: instead of a plain pass or fail, it says *by how much* the requirement held (positive) or failed (negative). That lets you compare and plot 30 runs rather than just counting failures. The project grades four requirements: stay safe, find each survivor, everyone eventually knows the liar, and cover 95% of the area.

## What the experiments ask

| Script | Question |
|---|---|
| `demo_once.py` | One seed, three fleets: the headline comparison |
| `exp_demo.py` | The same over many seeds (**not run yet**) |
| `exp_tradeoff.py` | How the audit fraction q trades completion time against missed survivors |

| Script | Question |
|---|---|
| `exp_knowledge_spread.py` | How fast news of the liar spreads, compared with an epidemic model |
| `exp_no_liar.py` | What caution costs when nobody lies |
| `exp_localisation.py` | W-MSR vs averaging for survivor positions |

The `aerial/lab/` folder holds the project's first, simpler 2D simulator. It's kept because the theory studies run there quickly: convergence speed vs graph connectivity ($\lambda_2$), stability limits on controller gains, W-MSR on random graphs, and how performance degrades as the number of liars grows.

## Tests

627 automated tests pass in about 20 s. The most meaningful ones:

- drones never import the ground-truth world;
- paired fleets really see identical randomness;
- the fast knowledge code agrees with a textbook possible-worlds implementation on small examples.

Most tests cover the lab and the STL engine. The v2 mission itself has only a handful, so avoid calling it "thoroughly tested".

## What's still shaky (be ready for these)

1. **The dramatic result needs the blunt liar.** With default settings on seed 0, naive found all 16 survivors with no collisions. The slide numbers use the blunt liar. Run many seeds in both settings and say clearly which is which.
2. **The oracle was the slowest fleet** on the default run (297 s, against 165 s for naive and 187 s for gated) and touched terrain once. As the supposed best case, that needs an explanation.
3. **The safety gate has a gap** when a possible liar's latest packet is lost: it skips that drone instead of using its last known distance. This is fixable in about an hour.
4. **The demo map never changes between seeds.** Multi-seed demo results vary radio and noise, not the city or which drone lies.
5. **The repository isn't ready to share.** There are no commits yet, and the course PDFs and lecture slides are staged for upload, which shouldn't go on a public GitHub.
6. **There is no written report yet.** The brief requires a PDF named after the group, with everyone's names, the GitHub link, and a last page of individual contributions. It is due 11 November at 23:59.